

Guía 3 ejercicios uso y aplicación colecciones en Python

Ejercicio 1: Análisis de pesos en salmonicultura

En el sur de Chile, la industria del salmón requiere monitorear constantemente el peso de los peces para tomar decisiones de alimentación y crecimiento.

Desarrolla un script en Python que permita analizar una muestra de salmones.

El programa debe:

- Generar una lista con **10 pesos de salmones** (puedes pedirlos al usuario o generarlos automáticamente usando random en un rango entre 2.5 y 6.0 kg).
- Mostrar:
 - El peso promedio (promedio = suma de pesos / cantidad de salmones)
 - El peso más bajo
 - El peso más alto
 - Cuántos salmones están sobre el promedio

Consideraciones:

- No usar funciones como max(), min() o sum()
- Usar ciclos (for o while)
- Separar el cálculo del promedio del análisis posterior

Ejercicio 2: Control de temperatura en invernadero agrícola

En la agricultura chilena, especialmente en zonas donde se producen hortalizas bajo invernadero, es importante controlar la temperatura para evitar pérdidas en los cultivos.

Desarrolla un script en Python que permita analizar las temperaturas registradas durante una jornada dentro de un invernadero.

El programa debe:

- Crear una **tupla** con 12 temperaturas registradas durante el día. Puedes ingresarlas manualmente o generarlas usando random en un rango entre 8 y 35 grados.
- Recorrer la tupla y mostrar:
 - La temperatura promedio.
 - La temperatura más baja.
 - La temperatura más alta.
 - Cuántas mediciones estuvieron bajo los 12 grados.
 - Cuántas mediciones estuvieron sobre los 30 grados.

Consideraciones:

- Usa una **tupla** porque las mediciones ya fueron registradas y no deberían modificarse.
- Se recomienda calcular manualmente el promedio, mínimo y máximo para practicar recorridos.
- El programa debe mostrar un pequeño mensaje final, por ejemplo: “Jornada estable”, “Riesgo por frío” o “Riesgo por exceso de calor”.

Ejercicio 3: Producción diaria en planta lechera

En una planta lechera del sur de Chile, se registra diariamente la cantidad de litros de leche entregados por distintos productores.

Desarrolla un script en Python que permita analizar la producción del día.

El programa debe:

- Crear una **lista de tuplas**, donde cada tupla tenga el formato: (nombre_productor, litros_entregados).
- Debes trabajar con al menos **8 productores** (puedes generarlos manualmente o con valores aleatorios usando random entre 100 y 1000 litros).
- Mostrar:
 - El total de litros recolectados en el día.
 - El productor que entregó más leche.
 - El productor que entregó menos leche.
 - Cuántos productores entregaron más de 500 litros.

Consideraciones:

- Se recomienda recorrer la estructura para calcular los resultados (evitar resolver todo directo con funciones).
- Debes acceder correctamente a los elementos de cada tupla (nombre y litros).
- Piensa cómo vas a comparar valores mientras recorres la lista.

Ejercicio 4: Registro de ventas en feria agrícola

En una feria agrícola local, distintos productores venden frutas y verduras durante la mañana.

El encargado necesita registrar las ventas realizadas para conocer el comportamiento del día.

Desarrolla un script en Python que permita registrar ventas hasta que el usuario decida terminar.

El programa debe:

- Usar un **diccionario** donde la clave sea el nombre del producto y el valor sea la cantidad total vendida.
- Solicitar repetidamente al usuario:
 - nombre del producto vendido
 - cantidad vendida
- Si el producto ya existe en el diccionario, se debe sumar la nueva cantidad a la cantidad anterior.
- El ingreso de ventas debe continuar hasta que el usuario escriba "fin".
- Al finalizar, mostrar:
 - todos los productos vendidos con sus cantidades
 - el producto más vendido
 - la cantidad total de unidades vendidas entre todos los productos

Consideraciones:

- Este ejercicio debe usar un ciclo de duración desconocida, por ejemplo while.
- Dentro del while, se puede recorrer el diccionario con for para obtener los resultados finales.
- Se recomienda calcular manualmente el producto más vendido para practicar comparación y recorrido.

Ejercicio 5: Monitoreo de niveles de oxígeno en centros de cultivo

En los centros de cultivo de salmones, el nivel de oxígeno disuelto en el agua es crítico para la supervivencia de los peces. Durante el día, un sensor registra mediciones periódicas.

Desarrolla un script en Python que permita analizar estas mediciones.

El programa debe:

- Generar una **lista** con valores de oxígeno disuelto (puedes usar random en un rango entre 4.0 y 10.0 mg/L o ingresarlos manualmente).
- La cantidad de mediciones no es fija: el usuario debe indicar cuántas mediciones se registrarán.
- Mostrar:
 - El promedio de oxígeno (promedio = suma de mediciones / cantidad de mediciones)
 - Cuántas mediciones están bajo 5.0 mg/L (nivel crítico)
 - Cuántas están entre 5.0 y 7.0 mg/L (nivel aceptable)
 - Cuántas están sobre 7.0 mg/L (nivel óptimo)

Consideraciones:

- Debes usar un ciclo para generar o ingresar las mediciones (por ejemplo, un while o for según corresponda).
- Luego, debes recorrer la lista para realizar el análisis.
- Piensa cómo clasificar cada medición en su categoría correspondiente.

Ejercicio 6: Control de asistencia de trabajadores en faena minera

En una faena minera del norte de Chile, se necesita registrar la asistencia diaria de los trabajadores para efectos de control operativo.

Desarrolla un script en Python que permita gestionar esta información.

El programa debe:

- Registrar trabajadores usando un **diccionario**, donde:
 - clave = nombre del trabajador
 - valor = estado de asistencia ("presente" o "ausente")
- Permitir ingresar trabajadores de forma repetitiva hasta que el usuario escriba "fin".
- Validar que el estado ingresado sea solo "presente" o "ausente".

Al finalizar, el programa debe mostrar:

- Cantidad total de trabajadores registrados
- Cantidad de trabajadores presentes
- Cantidad de trabajadores ausentes
- Lista de trabajadores ausentes

Consideraciones:

- Usar un ciclo de duración desconocida (while) para el ingreso de datos.
- Usar un for para recorrer el diccionario y hacer los conteos.
- Separar claramente la etapa de registro y la etapa de análisis.

Ejercicio 7: Control de stock en distribuidora de gas

Una distribuidora de gas necesita llevar el control del stock de cilindros durante la jornada. Existen distintos formatos de cilindros (por ejemplo: 5kg, 11kg, 15kg, 45kg), y durante el día se registran ventas y reposiciones.

Desarrolla un script en Python que permita gestionar esta situación.

El programa debe:

- Inicializar los tipos de cilindros disponibles y su stock inicial.
- Permitir al usuario, mediante un menú repetitivo:
 1. Ver stock actual
 2. Registrar venta de cilindros
 3. Registrar reposición de cilindros
 4. Ver historial de movimientos
 5. Salir
- Cada vez que se registre una venta o reposición, se debe guardar el movimiento realizado.
- Al finalizar el programa, se debe mostrar:
 - El stock final de cada tipo de cilindro
 - El tipo de cilindro con menor stock
 - La cantidad total de movimientos realizados durante el día

Consideraciones:

- El programa debe mantenerse en ejecución hasta que el usuario decida salir (ciclo de duración desconocida).
- Se deben recorrer estructuras de datos para:
 - Mostrar el stock
 - Analizar cuál tiene menor cantidad
 - Mostrar el historial de movimientos
- Validar que no se puedan realizar ventas si no hay suficiente stock.
- El estudiante debe decidir qué estructuras de datos utilizar para representar:
 - los tipos de cilindros
 - el stock
 - los movimientos

Ejercicio 8: Planificación de entregas en empresa de agua purificada

Una empresa de agua purificada realiza entregas diarias a distintos clientes de la ciudad. Cada entrega tiene un sector, una cantidad de botellones y un estado, por ejemplo: “pendiente”, “entregado” o “con deuda”.

Desarrolla un script en Python que permita registrar y analizar las entregas del día.

El programa debe:

- Permitir registrar entregas de forma repetitiva hasta que el usuario decida terminar.
- Para cada entrega, se debe ingresar:
 - nombre del cliente
 - sector
 - cantidad de botellones
 - estado de la entrega
- Al finalizar, mostrar:

- cantidad total de entregas registradas
- cantidad total de botellones entregados
- cantidad de entregas por estado
- sectores donde se realizaron entregas
- cliente con mayor cantidad de botellones solicitados

Consideraciones:

- El programa debe usar un ciclo de duración desconocida para registrar entregas.
- Se deben recorrer las estructuras de datos para calcular los resultados finales.
- El estudiante debe decidir cómo representar cada entrega y cómo almacenar el conjunto de entregas.
- Validar que la cantidad de botellones sea mayor que cero.
- Validar que el estado ingresado corresponda a una opción permitida.

Ejercicio 9: Registro de capturas en pesca artesanal

En una caleta del sur de Chile, los pescadores registran sus capturas diarias indicando el tipo de recurso extraído (por ejemplo: merluza, congrio, luga, erizo) y la cantidad obtenida en kilos.

Desarrolla un script en Python que permita registrar y analizar las capturas del día.

El programa debe:

- Permitir ingresar capturas de forma repetitiva hasta que el usuario indique que desea finalizar.
- En cada registro se debe ingresar:
 - nombre del recurso
 - cantidad capturada en kilos
- Si un recurso ya fue registrado anteriormente, se debe acumular su cantidad total.

Al finalizar, el programa debe mostrar:

- Total de kilos capturados en el día
- Cuántos tipos de recursos distintos fueron capturados
- El recurso con mayor cantidad capturada
- Un listado de todos los recursos con sus respectivos totales

Consideraciones:

- El registro debe realizarse con un ciclo de duración desconocida (while).
- Se deben recorrer las estructuras para calcular los resultados finales.
- El estudiante debe decidir cómo almacenar la información de las capturas.
- Validar que las cantidades ingresadas sean mayores que cero.
- Pensar cómo acumular valores cuando un recurso se repite.

Ejercicio 10: Sistema de operación diaria en centro de distribución

Un centro de distribución en Chile gestiona diariamente pedidos de distintos clientes. Cada pedido incluye productos, cantidades y un estado (por ejemplo: “pendiente”, “despachado”, “entregado”).

Desarrolla un script en Python que simule la operación de un día completo.

El programa debe:

- Permitir registrar pedidos de forma repetitiva hasta que el usuario decida finalizar.
- Para cada pedido se debe ingresar:
 - nombre del cliente
 - lista de productos (pueden ser varios por pedido)
 - cantidad por producto
 - estado del pedido
- Durante el registro, el sistema debe permitir:
 - agregar múltiples productos a un mismo pedido
 - registrar múltiples pedidos en el día

Al finalizar, el programa debe mostrar:

- Cantidad total de pedidos registrados
- Cantidad total de productos despachados (sumando todas las cantidades)
- Cantidad de pedidos por estado
- Cliente con mayor cantidad total de productos solicitados
- Listado completo de pedidos con su detalle

Consideraciones:

- El registro de pedidos debe realizarse con un ciclo de duración desconocida (while).
- Para agregar múltiples productos a un pedido, se debe utilizar otro ciclo anidado.
- Se deben recorrer estructuras para realizar los cálculos finales.
- El estudiante debe decidir cómo representar:
 - cada pedido
 - los productos dentro de un pedido
 - el conjunto total de pedidos
- Validar que las cantidades ingresadas sean mayores que cero.
- Pensar en cómo acumular información por cliente para el análisis final.